

EDL → Akamai Client List Connector

Test-Account Setup & Run Guide

Step-by-step: configure the connector to populate an existing Akamai Client List (“Ben’s IP Blacklist”) on a test account, including the firewall whitelist.

Version 1.0 — 13 July 2026 | Standard-library Python connector

1. What this does, and one hard rule

The connector reads the EDL IOC Manager's `Block_IoC_IPs.txt` and pushes the IP/CIDR entries into an **existing** Akamai **Client List**, then activates it, so App & API Protector (AAP/ASM) enforces them. It is outbound-only, standard-library only, and multi-tenant (each customer account is one tenant).

IMPORTANT: This tool **never creates** a Network List or Client List. It only updates a list that already exists. If you point it at a list name that doesn't exist, it stops with a clear error rather than creating anything — so the target list (e.g. "Ben's IP Blacklist") must be created in Akamai Control Center first (Step 4).

The setup, end to end: whitelist one hostname on the firewall (Step 2) → generate an Akamai API credential on the test account (Step 3) → create the Client List and wire its DENY into policy (Step 4) → place the files on the host (Step 5) → edit the config (Step 6) → run offline → dry-run → push (Step 7) → schedule it (Step 8).

2. Firewall — the exact hostname to whitelist

The script makes an **outbound** HTTPS connection from the host to Akamai. There is nothing inbound to open. The destination is the per-credential API hostname — the **exact value on the** host **line of your** `.edgerc`, which looks like:

WHITELIST THIS: `akab-XXXXXXXXXXXXXXXX-XXXXXXXXXXXXXXXX.luna.akamaiapis.net` — outbound **TCP 443** only. Copy the literal value from your `.edgerc`. If your firewall's FQDN objects struggle with the many short-TTL A records, use the wildcard `*.luna.akamaiapis.net`.

Field	Value
Direction	Outbound / egress (host → Akamai). No inbound rule.
Destination	The host from <code>.edgerc</code> , e.g. <code>akab-....luna.akamaiapis.net</code> (fallback: <code>*.luna.akamaiapis.net</code>)
Port	TCP 443 (HTTPS) only
Match by	FQDN, not IP — Akamai edge IPs rotate and are not published; an IP allowlist will break
Also needed	DNS resolution for that FQDN from the host (FQDN rules depend on it)

NOTE: If your egress passes through a **TLS-inspection proxy**, it re-signs Akamai's certificate with a corporate CA. The connector handles this — see Step 7 (`akamai_ca_bundle`). If egress uses an **explicit proxy**, set an `HTTPS_PROXY` environment variable on the task.

3. Generate the Akamai API credential (test account)

Do this while logged in to Akamai Control Center as an admin, **switched into the TEST account** (an API client is anchored to whatever account you are in when you create it).

#	Step
3.1	Use the account selector to switch into the test account .
3.2	Open  → ACCOUNT ADMIN → Identity & access , then Create API client .
3.3	Choose Advanced (not Quick) so you can scope the client.
3.4	Group: pick the group that contains the Client List you will manage (narrowest that still contains it).
3.5	APIs: search " client list ", add that service, set it to READ-WRITE (needed to add items and activate). Leave everything else off. Do NOT pick "Network Lists" — that is a different product.
3.6	Leave "manage multiple accounts" unticked for a test-only client.
3.7	Click Create API client , then Download the credentials. The <code>client_secret</code> is shown only once — save it now.

You end up with four values. Save them as a file named exactly `.edgerc` with a `[default]` section:

```
[default]
host = akab-xxxxxxxxxxxxxxxx-xxxxxxxxxxxxxxxx.luna.akamaiapis.net
client_token = akab-xxxxxxxxxxxxxxxx-xxxxxxxxxxxxxxxx
client_secret = xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx=
access_token = akab-xxxxxxxxxxxxxxxx-xxxxxxxxxxxxxxxx
```

NOTE: If you don't see **Create API client**, you lack API-provisioning rights — a Primary Admin must grant them or create the client for you. A client can only be granted APIs already enabled on the account, so confirm **Client Lists** is enabled on the test account.

4. Create the Client List & wire the DENY (one-time, in Control Center)

Because the connector never creates lists, create the target list yourself first:

#	Step
4.1	In the test account, create a Client List of type IP named exactly Ben's IP Blacklist (the connector matches on the exact name). It can start empty.
4.2	In your App & API Protector security policy, reference that Client List in the IP/Geo controls with a DENY action. This is what actually blocks traffic — the connector only manages the list's membership, not the policy.
4.3	Note: the connector finds the list by name, so you don't need to copy its ID. If two lists ever share the name, the connector will ask you to set an explicit <code>client_list_id</code> to disambiguate.

IMPORTANT: Enforcement is only real once the list is **referenced in the policy with DENY**. A list the policy doesn't reference blocks nothing, even when the connector fills it correctly.

5. Place the files on the host

#	Step
5.1	Unzip the connector to a folder outside any web root , e.g. <code>C:\Tools\edl-akamai-connector</code> , so its source and credentials are never web-served.
5.2	Put your .edgerc in that same folder (next to <code>connector.py</code>). The connector picks it up automatically — no config path needed . (To keep it elsewhere, set <code>"edgerc"</code> in the config instead.)
5.3	Lock the .edgerc ACL: remove inheritance; grant Read only to Administrators, SYSTEM, and the account the task runs as. It holds a live API secret — same discipline as any password.
5.4	Make sure the host has Python 3.9+ installed and can read the EDL data folder (<code>Block_IOC_IPs.txt</code>).

TIP: Answer to “does the `.edgerc` just go in the connector directory?” — **yes**. Drop it next to `connector.py` and it works with zero config. Setting an `edgerc` path in the config is only for keeping it somewhere else.

6. Configure config.test.json

Copy `config.test.example.json` to `config.test.json` and set the two paths. For a credential scoped to the test account, leave `account_switch_key` empty.

```
{
  "central_ioc_source": "C:\\inetpub\\edl-data\\Block_IOC_IPs.txt",
  "activation_network": "STAGING",
  "akamai_verify_ssl": true,
  "tenants": [
    {
      "name": "TestAccount-Ben",
      "enabled": true,
      "account_switch_key": "",
      "client_list_name": "Ben's IP Blacklist",
      "activation_network": "STAGING",
      "safety_shrink_pct": 30
    }
  ]
}
```

Field	Meaning
<code>central_ioc_source</code>	Path to <code>Block_IOC_IPs.txt</code> . For the very first push, you may point this at a small test <code>.txt</code> (a few IPs) so the result is easy to eyeball.
<code>client_list_name</code>	The exact list name to populate. Resolved to its ID at run time; never created.

Field	Meaning
account_switch_key	Empty when the .edgerc is scoped to the test account. (For managed accounts later, use a user-based API client + the switch key.)
activation_network	STAGING first; switch to PRODUCTION only after you confirm STAGING.
safety_shrink_pct	Abort (don't push) if the new set would delete more than this % of the live list — guards against a broken source wiping the list.

7. Run it — offline, then dry-run, then push

From the connector folder, in order. Each step is safe up to the last one.

7.1 Offline (no network) — proves parsing

```
python connector.py --config config.test.json --offline
```

Reads and parses the IP file, prints the counts. Makes no Akamai calls. Confirms the file path and that your IPs parse.

7.2 Dry-run (read-only Akamai call) — proves credential + firewall + list

```
python connector.py --config config.test.json --dry-run
```

Makes one **read-only** call: it resolves “Ben’s IP Blacklist” to its ID, reads the current entries, and prints the exact add/remove diff — but writes nothing. If this succeeds, your firewall whitelist, .edgerc, and the list name are all correct. A clean line looks like:

```
[--] TestAccount-Ben dry-run desired=3105 current=0 (+3105/-0) no writes
```

7.3 Push (writes + activates on STAGING)

```
python connector.py --config config.test.json
```

Pushes the add/remove delta into the list and activates it on **STAGING**. Then verify in Control Center that “Ben’s IP Blacklist” shows the entries and the staging activation succeeded, and test that a listed IP is blocked. Only then set `activation_network` to `PRODUCTION` and run again.

NOTE: A non-zero exit code means a tenant aborted or errored — point your scheduler’s alerting at it. The connector isolates tenants, so one failure does not stop the others.

7.4 If the dry-run fails with a certificate error

If you see `CERTIFICATE_VERIFY_FAILED`, a TLS-inspection proxy is re-signing the connection. The message tells you the fix. Preferred: set `akamai_ca_bundle` to your corporate root CA PEM (keeps verification on). Last resort: bypass with `--insecure` (or `"akamai_verify_ssl": false`) — it prints a security warning because, with verification off, an interceptor could tamper with the responses this tool acts on.

8. Schedule it (Windows Task Scheduler)

Nothing is configured in IIS Manager — the connector is a standalone script, not an IIS site. Create a scheduled task:

Tab	Setting
General	Run as a dedicated service account; “Run whether user is logged on or not.” The account needs Read on the EDL data folder and the .edgerc, and “Log on as a batch job” rights.
Trigger	Your cadence (e.g. hourly) — how fresh you want Akamai to be.
Action	Program = <code>python.exe</code> ; Arguments = <code>connector.py --config config.test.json</code> ; Start in = the connector folder.
Settings	“If the task fails, restart.” Watch the task’s Last Run Result (the connector’s exit code) for alerting.

9. Troubleshooting

Symptom	Cause → fix
“No existing IP Client List named ...”	The list doesn’t exist (the tool never creates one) → create it in Control Center (Step 4), matching the name exactly.

Symptom	Cause → fix
"Multiple Client Lists named ..."	Two lists share the name → set an explicit client_list_id in the tenant to disambiguate.
CERTIFICATE_VERIFY_FAILED	Corporate TLS-inspection proxy → set akamai_ca_bundle (recommended) or --insecure (last resort). See 7.4.
Connection timed out / cannot resolve host	Firewall egress not open, or DNS → whitelist outbound 443 to the .edgerc host / *.luna.akamaiapis.net (Step 2); confirm DNS.
HTTP 403	The API client lacks the Client List READ-WRITE grant, or the group doesn't contain the list → fix the client's permissions/group (Step 3).
Fetch succeeds but nothing is blocked	The list isn't referenced in the AAP policy with DENY → Step 4.2.
accountSwitchKey error	Account switching doesn't work with a service-account API client → use a normal user-based client (only relevant once you go multi-account).

This guide covers the test-account path. For the architecture, security model, and the full multi-tenant/subscription configuration, see the connector's README and the project's other documentation.